

Lab Report: Two Pointers and Sliding Window

Student: Cao Bao Khuong
ID: ITDSIU22176

Part B: Classical Exercises

Exercise B1: Two Sum II (Sorted Array)

- **Problem Restatement:** Given a 1-indexed array of integers already sorted in non-decreasing order, find two numbers such that they add up to a specific target number.
- **Pattern:** Two Pointers (Opposite Direction).
- **Invariant:** At any step, if a solution exists, it must be between the `left` and `right` indices.
- **Brute-force Idea:** Use nested loops to check every possible pair (i, j) . This takes $O(n^2)$ time.
- **Optimized Idea:** Start with two pointers at the ends. If their sum is smaller than the target, move the left pointer forward. If larger, move the right pointer backward.
- **Complexity Analysis:**
 - **Time:** $O(n)$ - Each element is visited at most once.
 - **Space:** $O(1)$ - No extra data structures used.
- **Test Evidence:**
 - `[2, 7, 11, 15]`, `target 9` -> Output: `[1, 2]` (Correct)
 - `[2, 3, 4]`, `target 6` -> Output: `[1, 3]` (Correct)
 - `[-1, 0]`, `target -1` -> Output: `[1, 2]` (Edge Case: Negative)

Exercise B2: Remove Duplicates from Sorted Array

- **Problem Restatement:** Remove duplicates in-place from a sorted array such that each unique element appears only once. Return the new length.
- **Pattern:** Two Pointers (Read/Write).
- **Invariant:** All elements at or before the `write` pointer are unique and in sorted order.
- **Brute-force Idea:** Create a new array and only copy unique elements, then copy back. This uses $O(n)$ space.
- **Optimized Idea:** Use a `write` pointer to keep track of the last unique element found and a `read` pointer to scan the array.
- **Complexity Analysis:**
 - **Time:** $O(n)$ - Single pass through the array.
 - **Space:** $O(1)$ - In-place transformation.
- **Test Evidence:**
 - `[1, 1, 2]` -> New Length: 2 (Correct)
 - `[0, 0, 1, 1, 1, 2, 2, 3, 3, 4]` -> New Length: 5 (Correct)

- [1] -> New Length: 1 (Edge Case: Single element)

Exercise B3: Move Zeroes

- **Problem Restatement:** Move all 0's to the end of an array while maintaining the relative order of the non-zero elements.
- **Pattern:** Two Pointers (Same Direction).
- **Invariant:** Elements before the **slow** pointer are all non-zero and in their original relative order.
- **Brute-force Idea:** Create a new array, fill it with non-zero elements first, then fill the rest with zeroes. $O(n)$ space.
- **Optimized Idea:** Iterate with a **fast** pointer; whenever a non-zero is found, swap it with the element at the **slow** pointer.
- **Complexity Analysis:**
 - **Time:** $O(n)$ - Total number of operations is linear.
 - **Space:** $O(1)$ - Swapping is done in-place.
- **Test Evidence:**
 - [0, 1, 0, 3, 12] -> Result: [1, 3, 12, 0, 0] (Correct)
 - [0] -> Result: [0] (Edge Case: Only zero)
 - [4, 2, 4, 0, 3, ...] -> Result: [4, 2, 4, 3, 5, 1, 0, 0, 0, 0] (Correct)

Exercise B4: Maximum Sum Subarray of Size k

- **Problem Restatement:** Find the maximum sum of any contiguous subarray of a fixed size k .
- **Pattern:** Fixed-size Sliding Window.
- **Invariant:** The window sum always represents the sum of k consecutive elements.
- **Brute-force Idea:** For every possible starting index i , calculate the sum of elements from i to $i + k - 1$. $O(n \cdot k)$ time.
- **Optimized Idea:** Maintain a running sum. When moving the window, subtract the element leaving the window and add the one entering.
- **Complexity Analysis:**
 - **Time:** $O(n)$ - Each element is added and subtracted exactly once.
 - **Space:** $O(1)$.
- **Test Evidence:**
 - [2, 1, 5, 1, 3, 2], $k=3$ -> Result: 9 (Correct)
 - [2, 3, 4, 1, 5], $k=2$ -> Result: 7 (Correct)
 - [1, 2, 3], $k=1$ -> Result: 3 (Edge Case: $k=1$)

Exercise B5: Minimum Size Subarray Sum

- **Problem Restatement:** Find the minimal length of a contiguous subarray of which the sum is $\geq$ target.
- **Pattern:** Variable-size Sliding Window.

- **Invariant:** The current window `[left, right]` has a sum that we try to keep at least as large as the target while minimizing length.
- **Brute-force Idea:** Check all possible subarrays (i, j) and find the minimum length sum $\geq$ target. $O(n^2)$ time.
- **Optimized Idea:** Expand the `right` pointer to increase sum. Once sum $\geq$ target, shrink from `left` as much as possible while maintaining the condition.
- **Complexity Analysis:**
 - **Time:** $O(n)$ - Each pointer moves from left to right once.
 - **Space:** $O(1)$.
- **Test Evidence:**
 - target 7, [2, 3, 1, 2, 4, 3] -> Result: 2 (Correct)
 - target 4, [1, 4, 4] -> Result: 1 (Correct)
 - target 11, [1, 1, 1, ...] -> Result: 0 (Edge Case: No solution)

Exercise B6: Longest Substring Without Repeating Characters

- **Problem Restatement:** Given a string, find the length of the longest substring without repeating characters.
 - **Pattern:** Variable-size Sliding Window (State Tracking).
 - **Invariant:** The window `[left, right]` always contains unique characters.
 - **Brute-force Idea:** Check all possible substrings and verify uniqueness for each. $O(n^3)$ or $O(n^2)$ time.
 - **Optimized Idea:** Use a Hash Set to track characters in the current window. If a duplicate is found at `right`, shrink from `left` until the duplicate is removed.
 - **Complexity Analysis:**
 - **Time:** $O(n)$ - Each character is added/removed once.
 - **Space:** $O(\min(n, m))$ - For the Hash Set.
 - **Test Evidence:**
 - "abcabcbb" -> Result: 3 (Correct)
 - "bbbbbb" -> Result: 1 (Correct)
 - "pwwkew" -> Result: 3 (Correct)
-

Part C: Intermediate Exercises

Exercise C1: Container With Most Water

- **Problem Restatement:** Find two lines that together with the x-axis form a container, such that the container contains the most water.
- **Pattern:** Two Pointers (Opposite Direction).
- **Invariant:** The `left` and `right` pointers always bound the potential maximum area.

- **Brute-force Idea:** Check all pairs of lines and calculate area. $O(n^2)$.
- **Optimized Idea:** Start from both ends. Move the pointer pointing to the shorter line inward, as this is the only way to potentially find a larger area.
- **Complexity Analysis:**
 - **Time:** $O(n)$.
 - **Space:** $O(1)$.
- **Test Evidence:**
 - [1, 8, 6, 2, 5, 4, 8, 3, 7] -> Result: 49 (Correct)
 - [1, 1] -> Result: 1 (Edge Case: Minimum lines)
 - [4, 3, 2, 1, 4] -> Result: 16 (Correct)

Exercise C2: 3Sum

- **Problem Restatement:** Find all unique triplets in the array which gives the sum of zero.
 - **Pattern:** Sorting + Two Pointers.
 - **Invariant:** After sorting, for each i , the Two Pointer search finds all pairs j, k such that $nums[i] + nums[j] + nums[k] = 0$.
 - **Brute-force Idea:** Three nested loops to check all triplets. $O(n^3)$.
 - **Optimized Idea:** Sort the array. Iterate through each element and use the Two Pointers technique (similar to Two Sum II) on the remaining part of the array.
 - **Complexity Analysis:**
 - **Time:** $O(n^2)$ - Sorting takes $O(n \log n)$, then n iterations of $O(n)$ two-pointer search.
 - **Space:** $O(\log n)$ or $O(n)$ depending on the sorting implementation.
 - **Test Evidence:**
 - [-1, 0, 1, 2, -1, -4] -> Result: [[-1, -1, 2], [-1, 0, 1]] (Correct)
 - [0, 1, 1] -> Result: [] (No solution)
 - [0, 0, 0] -> Result: [[0, 0, 0]] (Edge Case: All same)
-

Part D2: Explain Without Code (Exercise B1)

Exercise B1: Two Sum II (Sorted Array) The logic behind the Two Pointers approach for a sorted array is intuitive yet powerful. We start with two pointers at the extreme ends of the array—one at the smallest element and one at the largest. Because the array is sorted, we know that if we need a larger sum, we must move the left pointer to the right (to a larger number). Conversely, if the current sum is too high, the only way to potentially reach the target is to move the right pointer to the left (to a smaller number).

This strategy effectively "narrows down" the search space by eliminating millions of impossible pairs without ever having to look at them. We maintain the

invariant that the target pair (if it exists) must be within our current bounds. This reduces the complexity from a nested quadratic scan to a single linear pass.